

StockFlow SIR callbacks

Lockdown and vaccination interventions in a stock-and-flow SIR model

This vignette repeats the SIR intervention example with `StockFlow.jl`. It uses the same model as the AlgebraicPetri vignette and the parameterization from [epirecipes/sir-julia](https://epirecipes.github.io/sir-julia/): $S(0)=990$, $I(0)=10$, $R(0)=0$, contact rate $c=10$, transmission probability $\beta=0.05$, and recovery rate $\gamma=0.25$.

`StockFlow.jl` currently has a Julia compatibility bound of 1.9 – 1.11, so this vignette has its own local `Project.toml` and should be rendered with Julia 1.11.

```
using CategoricalInterventions
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots
using StockFlow
using StockFlow.Syntax

default(; linewidth=2, grid=false)

sir_model = @stock_and_flow begin
    :stocks
    S
    I
    R

    :parameters
    inf
    rec

    :dynamic_variables
    v_infection = inf * S * I
    v_recovery = rec * I
```

```

:flows
S => f_infection(v_infection) => I
I => f_recovery(v_recovery) => R

:sums
N = [S, I, R]
end

u0 = LVector(S=990.0, I=10.0, R=0.0)
p0 = LVector(inf=0.05 * 10.0 / sum(u0), rec=0.25)
tspan = (0.0, 40.0)
save_grid = 0.0:0.5:40.0

prob = ODEProblem(vectorfield(sir_model), u0, tspan, p0)
baseline = solve(prob, Tsit5(); saveat=save_grid)

baseline.retcode, baseline[3, end], sum(baseline.u[end])

```

(SciMLBase.ReturnCode.Success, 775.6835919143084, 999.9999999999998)

Interventions

The infection flow is a stock-and-flow arrow from S to I; the intervention targets the labelled rate parameter `:inf`. Vaccination is represented as a state transfer: remove 50 people from S and add 50 people to R at time 15.

```

indexing = PetriIndexing(parameters=[:inf, :rec], states=[:S, :I, :R])

inf_rate = Target(
    :inf;
    kind=Parameter,
    value_type=Float64,
    algebra=MultiplicativeAlgebra(),
)

lockdown = InterventionProgram(
    InterventionAtom(:lockdown, inf_rate, Interval(5.0, 25.0), Scale(0.5)),
)

susceptible = Target(:S; kind=State, value_type=Float64, algebra=AdditiveAlgebra())
recovered = Target(:R; kind=State, value_type=Float64, algebra=AdditiveAlgebra())

```

```

vaccination = InterventionProgram(
    InterventionAtom(:vaccinate_from_s, susceptible, Interval(15.0, 16.0), Add(-50.0)),
    InterventionAtom(:vaccinate_to_r, recovered, Interval(15.0, 16.0), Add(50.0)),
)

combined = compose_interventions(lockdown, vaccination)

```

InterventionProgram(3 atoms)

```

lockdown_sol = solve(
    prob,
    Tsit5();
    callback=to_callback(lockdown, indexing; baseline_p=p0),
    saveat=save_grid,
)

vaccination_sol = solve(
    prob,
    Tsit5();
    callback=to_callback(vaccination, indexing),
    saveat=save_grid,
)

combined_sol = solve(
    prob,
    Tsit5();
    callback=to_callback(combined, indexing; baseline_p=p0),
    saveat=save_grid,
)

(
    baseline_R = baseline[3, end],
    lockdown_R = lockdown_sol[3, end],
    vaccination_R = vaccination_sol[3, end],
    combined_R = combined_sol[3, end],
    combined_total = sum(combined_sol.u[end]),
)

```

(baseline_R = 775.6835919143084, lockdown_R = 354.0601008712393, vaccination_R = 773.72864779)

Cross-check against the same SIR equations

To check that this is the same SIR model as the AlgebraicPetri and AlgebraicDynamics versions, solve the equations directly and compare the saved trajectories. The maximum difference is at numerical solver tolerance.

```
function sir_ode(u, p, t)
    infection = p.inf * u.S * u.I
    recovery = p.rec * u.I
    return LVector(S=-infection, I=infection - recovery, R=recovery)
end

reference_prob = ODEProblem(sir_ode, u0, tspan, p0)

reference_baseline = solve(reference_prob, Tsit5(); saveat=save_grid)
reference_combined = solve(
    reference_prob,
    Tsit5();
    callback=to_callback(combined, indexing; baseline_p=p0),
    saveat=save_grid,
)

max_reference_difference = maximum(abs, Array(baseline) .- Array(reference_baseline))
max_combined_difference = maximum(abs, Array(combined_sol) .- Array(reference_combined))

max_reference_difference, max_combined_difference
```

```
(1.755369112288463e-7, 1.8250489119964186e-8)
```

```
max_reference_difference < 1e-6 && max_combined_difference < 1e-6
```

```
true
```

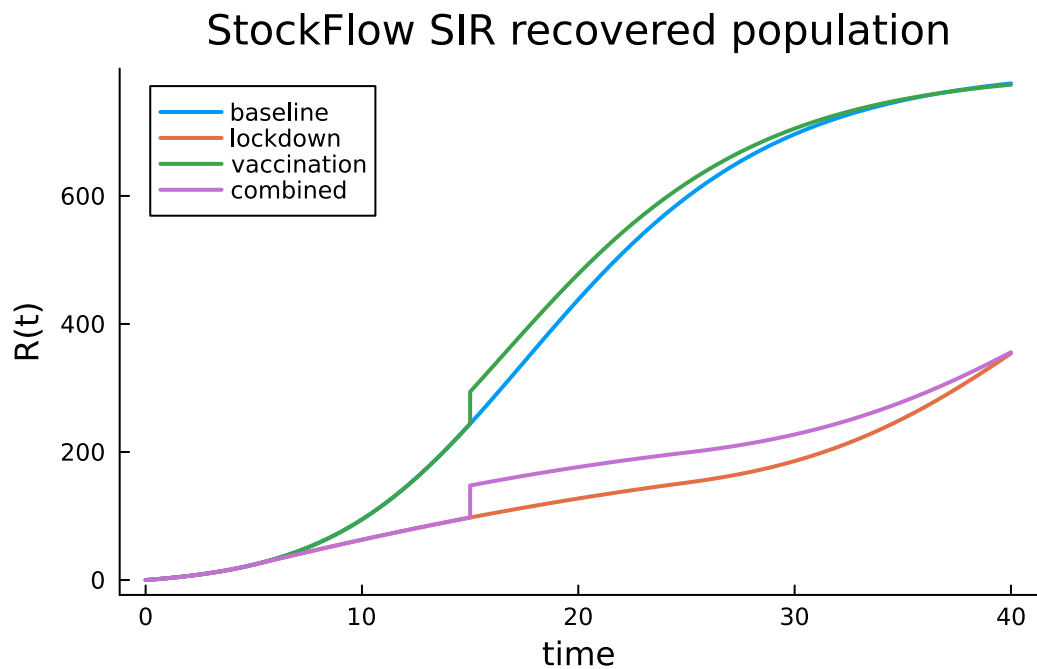
Plotting

```
plot(
    baseline.t,
    baseline[3, :];
    label="baseline",
```

```

    xlabel="time",
    ylabel="R(t)",
    title="StockFlow SIR recovered population",
)
plot!(lockdown_sol.t, lockdown_sol[3, :]; label="lockdown")
plot!(vaccination_sol.t, vaccination_sol[3, :]; label="vaccination")
plot!(combined_sol.t, combined_sol[3, :]; label="combined")

```



```

plot(
    combined_sol.t,
    combined_sol[1, :];
    label="S",
    xlabel="time",
    ylabel="population",
    title="StockFlow combined lockdown and vaccination",
)
plot!(combined_sol.t, combined_sol[2, :]; label="I")
plot!(combined_sol.t, combined_sol[3, :]; label="R")

```

StockFlow combined lockdown and vaccination

